

AI GitHub Agent System

Final Project Documentation

1. Project Overview

This project is an AI-powered agent system designed to understand, analyze, and interact with GitHub repositories. The system goes beyond a simple chatbot by combining Retrieval-Augmented Generation (RAG), Model Context Protocol (MCP), and agent-based reasoning to deliver meaningful actions like answering code-related questions, diagnosing bugs, and proposing fixes via draft pull requests.

Unlike traditional 'chat with codebase' demos, this system can plan tasks, invoke multiple tools, retrieve repository context, and apply structured validation before producing any output. It is split into an offline indexing pipeline and an online agent execution pipeline, which mirrors how real-world AI coding assistants are designed.

The final goal is to build an autonomous-but-safe AI software engineer that can reason about code, propose changes, and always defer the final merge decision to a human reviewer.

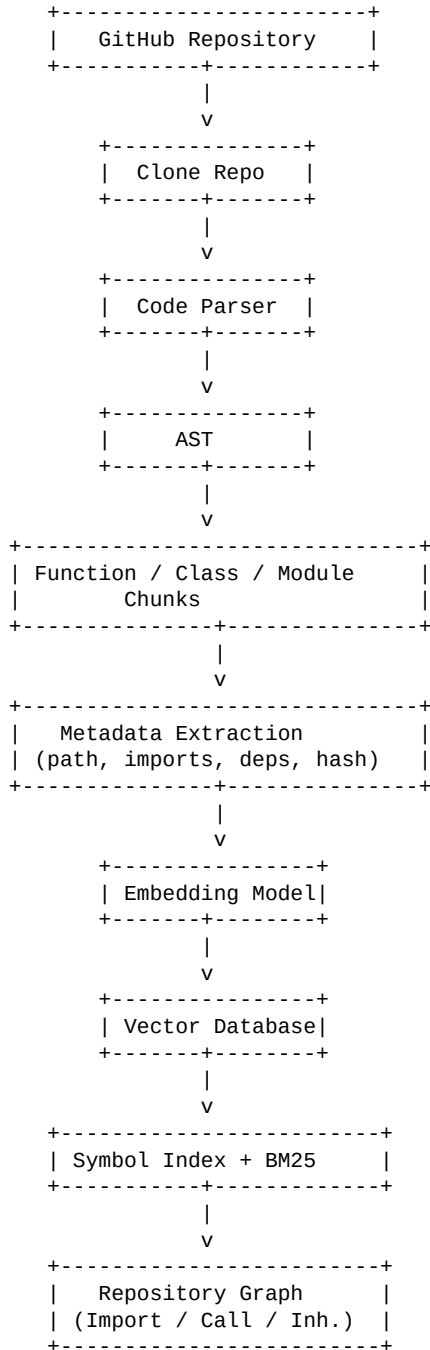
2. Project Brief (Core Components)

- **Indexing:** Repository is cloned, parsed via AST, split into function/class chunks, and stored with embeddings, symbol index, and a repository graph.
- **Q&A System:** User can ask questions about code, and the system retrieves relevant chunks to generate context-aware answers.
- **MCP–GitHub Bridge:** Connects the agent to live GitHub data such as files, issues, PRs, and commits using MCP-based tools.
- **Fix Proposal:** Given a task or bug, the agent reasons over retrieved code, generates a diff, and creates a draft pull request for human review.

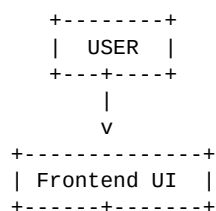
3. System Architecture

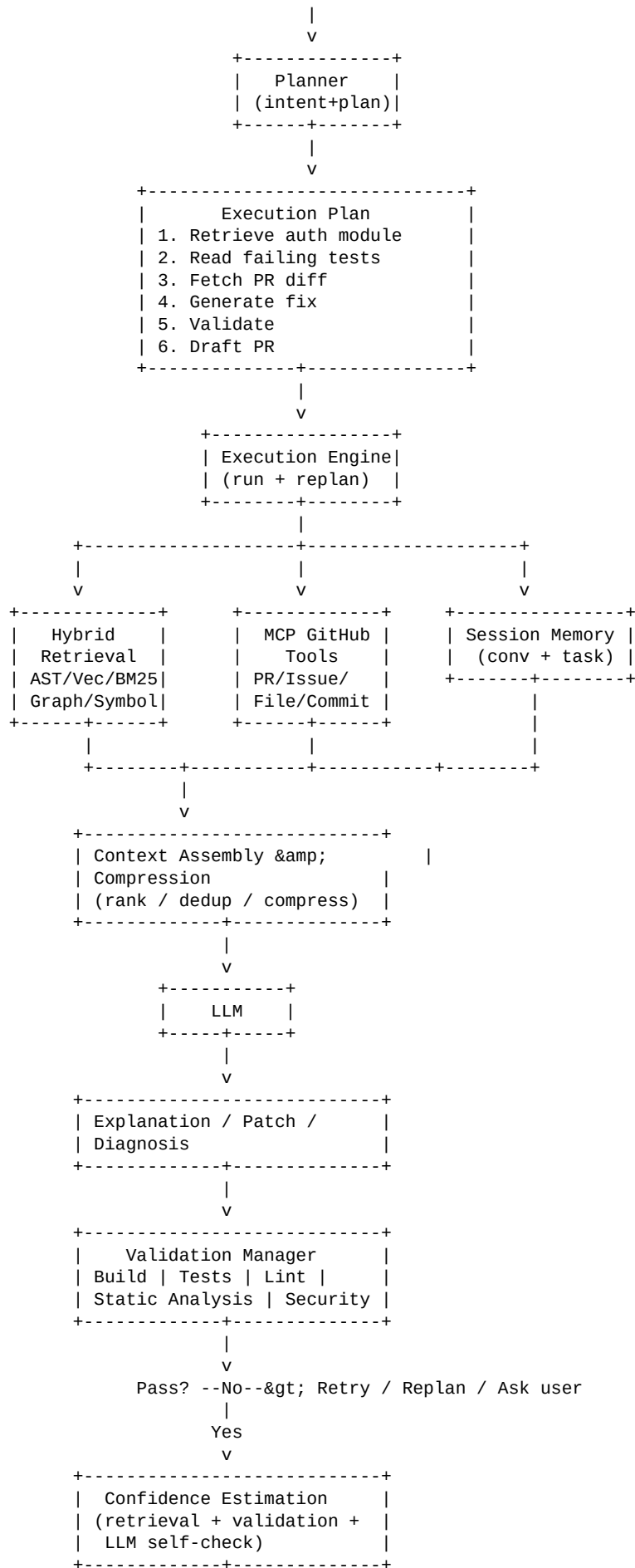
The system is divided into two distinct architectures for clarity and modularity:

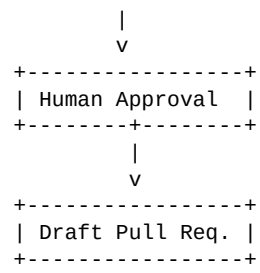
Part 1 — Offline Pipeline (Repository Indexing)



Part 2 — Online System (Runtime Agent Pipeline)

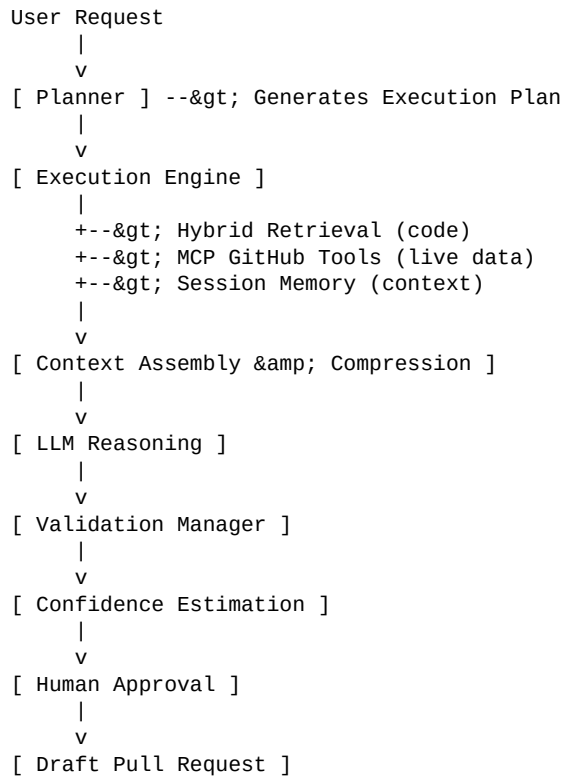






4. Simplified Flow Chart

This shows the end-to-end flow of how a user request is processed:



5. Key Components Explained

Planner: Understands the user's intent and generates a structured, multi-step execution plan instead of directly calling the LLM.

Execution Engine: Executes each step of the plan, invokes tools as needed, and replans if a step fails or produces unexpected results.

Hybrid Retrieval: Combines AST-based chunks, vector embeddings, BM25 keyword search, repository graph traversal, and symbol indexing for more accurate code retrieval.

MCP GitHub Tools: Allows the agent to fetch live repository data such as files, issues, pull requests, and commits using the Model Context Protocol.

Session Memory: Stores short-term conversation history, task progress, and retrieved context so multi-step workflows can continue smoothly.

Context Assembly & Compression: Ranks retrieved content, removes duplicates, and compresses it to fit within the LLM context window.

LLM: Performs reasoning, explanations, and code generation using the assembled context.

Validation Manager: Runs applicable checks such as build, unit tests, linting, static analysis, and security scans, and handles failure cases by retrying or replanning.

Confidence Estimation: Computes a confidence score based on retrieval relevance, validation results, tool success, and LLM self-check — preventing low-confidence fixes from being pushed.

Human Approval: Final safety layer where a developer reviews the proposed draft PR before merging.

6. Key Upgrades in This Architecture

- Removed unnecessary Intent Classification — Planner now handles intent inference directly.
- Planner produces a structured execution plan instead of being a vague 'decision box'.
- Added Execution Engine which actually runs the plan and supports replanning on failure.
- Hybrid Retrieval combining AST, embeddings, BM25, repository graph, and symbol index for code-aware search.
- Validation pipeline now has explicit failure handling (retry, replan, or ask user).
- Confidence estimation based on real evidence (retrieval, validation, LLM self-check) — not a random score.
- Session-scoped memory (no false claim of permanent long-term memory).
- Context Assembly & Compression layer added to handle large repositories within model context limits.
- Separated Offline Indexing Pipeline from Online Runtime Pipeline for cleaner system design.
- Replaced vague labels like 'AI Agent' with concrete components (Planner, Executor, Retriever, Validator).

7. Project Roadmap (Week-by-Week)

Phase	Goal
Week 1	Build the offline indexing pipeline: clone repo, parse with AST, chunk by function/class, extract meta
Week 2	Implement RAG-based Q&A system using retrieval to answer questions about the codebase.
Week 3	Integrate MCP server to fetch live repository data (files, issues, PRs, commits).
Week 4	Implement fix generation: agent reasons over retrieved code and produces a diff for a given task o
Week 5	Add validation pipeline (basic build/lint/tests) and draft PR creation via GitHub API with human app
Future	Hybrid retrieval upgrades, repository graph, replanning, evidence-based confidence estimation, an

8. Implemented vs Planned (Be Honest)

A key principle followed in this project is to never overclaim features. The architecture above describes the full design vision, but only some parts will be implemented in the first version. The remaining components are explicitly marked as planned enhancements.

Component	Status
Indexing (basic AST/embedding)	Implemented
RAG-based Q&A	Implemented
MCP GitHub integration	Implemented

Basic fix generation	Implemented
Draft PR creation	Implemented
Hybrid retrieval (BM25 + Graph + Symbol)	Planned
Execution Engine with replanning	Planned
Evidence-based confidence estimation	Planned
Full validation pipeline (build/tests/security)	Planned
Context compression layer	Planned

9. Summary (Spoken Version)

This project is an agent-based system that combines RAG, MCP, planning, hybrid retrieval, and a structured validation pipeline. It is divided into two parts: an offline pipeline that indexes the repository using AST parsing, embeddings, symbol indexing, and a repository graph; and an online pipeline where a planner generates an execution plan, an execution engine runs the steps using retrieval and GitHub tools, and the LLM produces explanations or code fixes. The output then passes through a validation manager (build, tests, lint, static analysis, security), a confidence estimation step, and human approval before creating a draft pull request. The architecture is designed to support advanced features such as hybrid retrieval, replanning, and confidence scoring, while the current implementation focuses on the core flow — and the rest is planned in future phases.

10. Final Rating

Category	Score
Architecture Quality	9.5 / 10
Engineering Depth	9.5 / 10
Portfolio Strength	10 / 10 (if core is implemented)
Interview Impression	9.5 / 10
Originality	8.5 / 10

Final Note

The strongest version of this project is not the most ambitious diagram — it is the one where every component on the diagram corresponds to working, explainable code. Implementing 80% of this architecture, while being transparent about what is built versus planned, will make this project stand out as a serious AI/LLM engineering effort.